

Глубинное обучение в обработке звука

Синтез речи II: *vocoders* и *end-to-end* модели

План лекции

- *GST* и *TP-GST*

- *Vocoders*

- *End-to-End* модели

Global Style Tokens, 2018

Проблемы прошлых моделей:

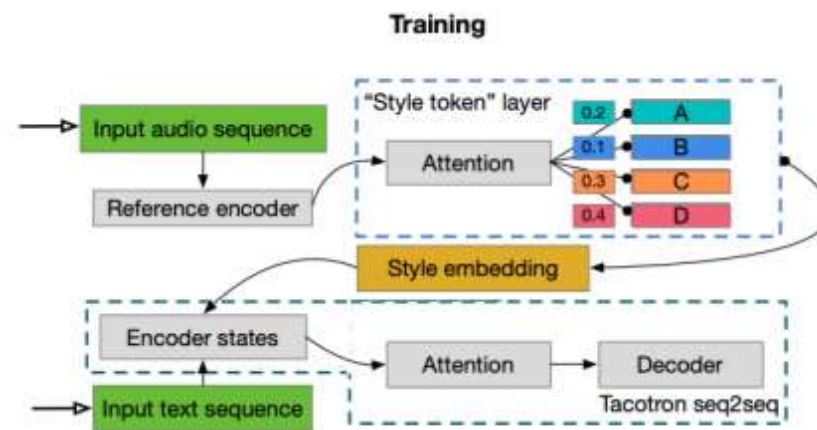
- Разметка эмоций и стилей очень дорогая
- Большинство *TTS* моделей выучивают плоскую, невыразительную речь
- Сложно управлять скоростью, интонациями и эмоциями

Идея:

- Добавим в модель векторы стиля, которые извлекают из аудио признаки и позволяют управлять ими

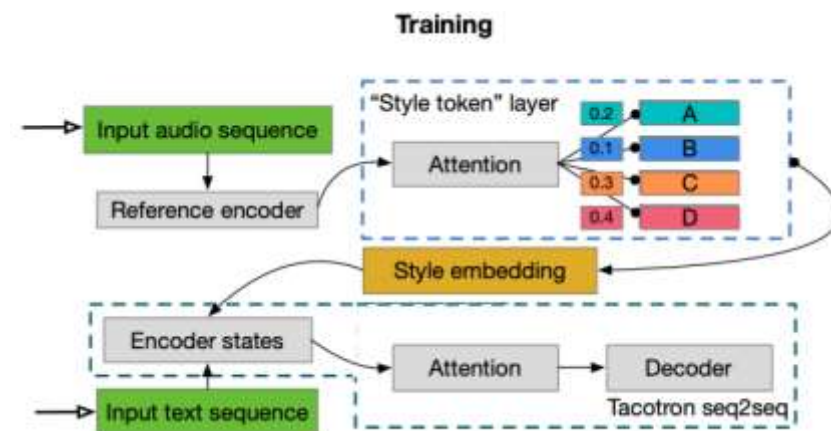
Архитектура GST

- Основная модель – *Tacotron*
- *Reference Encoder* сжимает аудио в вектор
- Основной блок – *Style token layer*, внутри которого находится словарь стилей
- *Attention* связывает входное аудио с векторами стиля



Обучение GST

- На вход подается пара «текст + оригинальное аудио»
- Аудио проходит через *Reference Encoder* и сопоставляется с набором векторов через *Attention*
- *Style Embedding* – взвешенная сумма векторов, которая служит условием для генерации
- Полученный стиль добавляется к тексту
- *Decoder* восстанавливает спектрограмму
- Вся система вместе с векторами обучается минимизировать ошибку восстановления звука



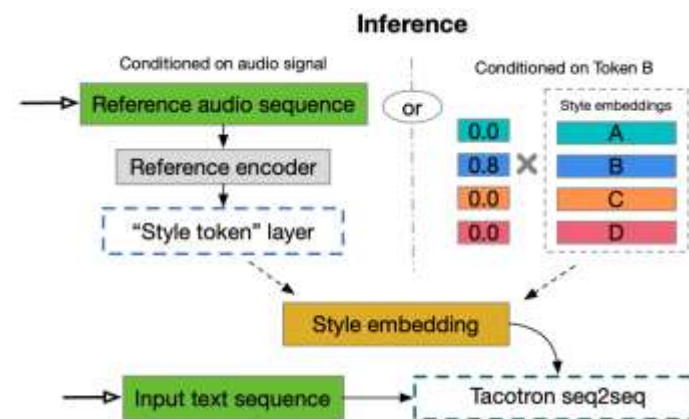
Генерация GST

Conditioned on audio signal:

- Подаем на вход аудио с некоторым стилем
- Получаем из него вектор стиля
- Это позволяет делать *Style Transfer*

Conditioned on Token:

- Убираем *Reference Encoder* и вручную задаём веса для векторов
- Генерируем речь с конкретным стилем, который выучил определенный вектор



Результаты GST

- Векторы являются интерпретируемыми
- Появилась возможность управлять стилем речи
- При обучении модель может научиться отделять речь от шумов

You are not so important after all, Pau Amma, he said



Reference



Baseline Tacotron



GST Tacotron

Text Predicted GST, 2018

Проблемы прошлых моделей:

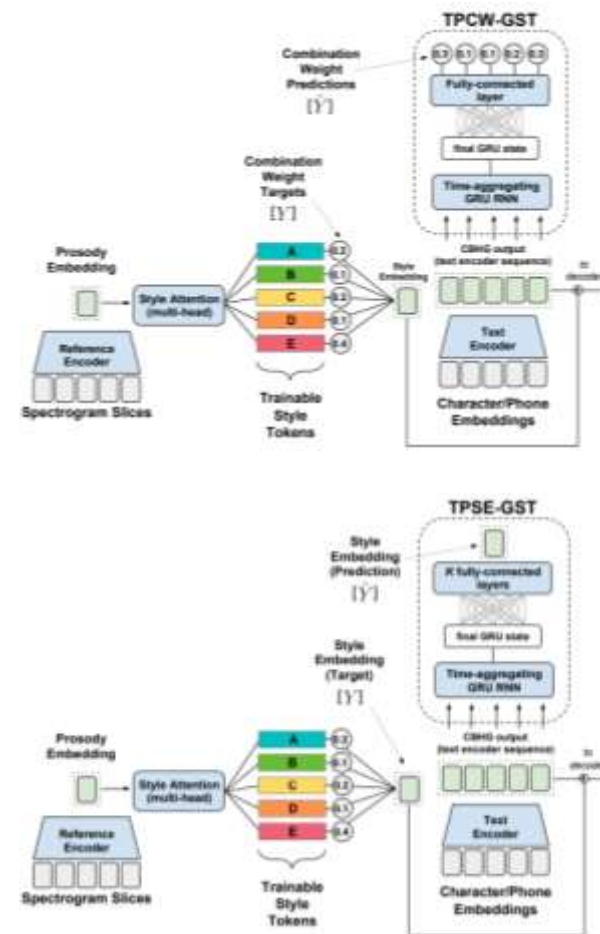
- Одному тексту может соответствовать множество вариантов произношения
- Обычные модели генерируют усредненную речь
- *GST* требует аудио или ручной выбор вектора для управления стилем

Идея:

- Обучим модель предсказывать вектор стиля напрямую из текста

Обучение и инференс $TP - GST$

- Основа *Tacotron* + *GST*
- *Reference Encoder* сжимает аудио в вектор стиля
- *Text Predictor* берёт выход текстового энкодера и пытается предсказывать вектор стиля
- *TPCW - GST* предсказывает процент смешивания векторов
- *TPSE - GST* предсказывает напрямую стилевой вектор



Vocoders

WaveNet, 2016

Проблемы параметрических моделей:

- Основаны на упрощенных моделях *НММ*, которые не соответствуют природе речи
- Звук часто сглаженный и содержит артефакты
- Используют вокодеры, что вносит потери в качестве

Проблемы конкатенативных моделей:

- Требуют огромную базу записанной речи
- Сложно менять голос или интонацию
- Есть артефакты при склеивании фрагментов

Идея:

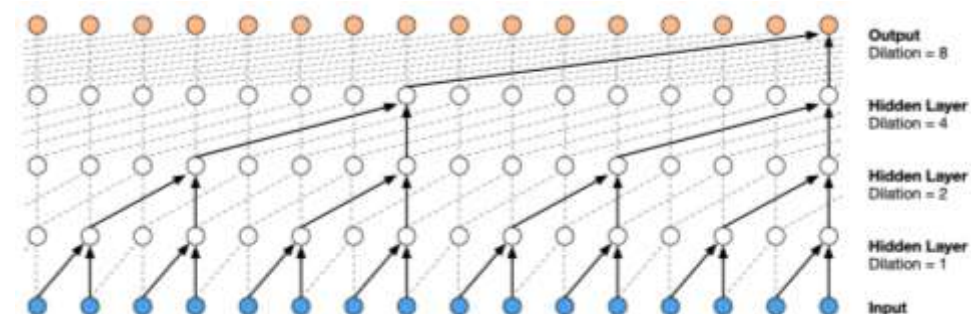
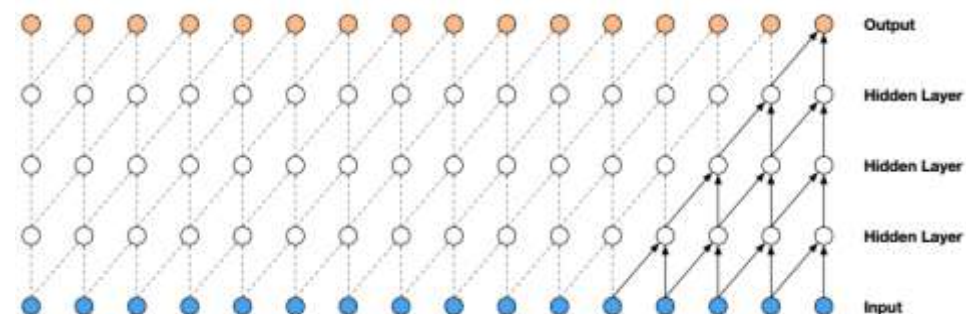
- Откажемся от промежуточных параметров
- Будем напрямую из текста генерировать аудио

Архитектура: *Causal* и *Dilated Convolutions*

Проблема:

- Аудио требует моделирования длинных зависимостей

- ***Causal Convolutions*** гарантируют, что модель не будет подглядывать в будущее
- Для захвата длинных зависимостей используются ***Dilated Convolutions***
- Коэффициент расширения растет экспоненциально с глубиной слоя



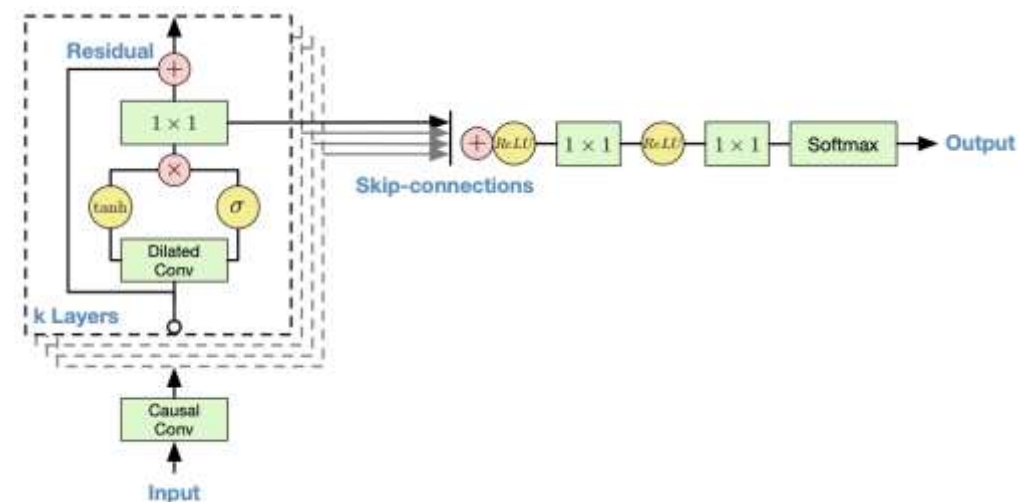
Архитектура WaveNet

- За основу взята архитектура *PixelCNN*

- Вместо *ReLU* используется *Gated Activation Units*:

$$z = \tanh(W_{f,k} * x) \odot \sigma(W_{g,k} * x)$$

- Используются *residual* и *skip connections* для стабильности обучения



μ -law

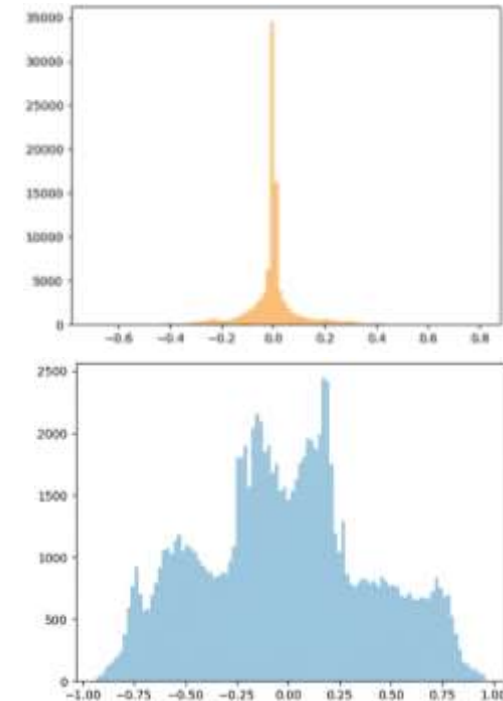
- В сыром аудио $2^{16} = 65536$ возможных значений амплитуды
- Делать предсказания на 65536 классов затратно

Идея:

- Авторы предложили использовать нелинейное квантование по μ -закону для сжатия диапазона до 256 классов:

$$f(x_t) = \text{sign}(x_t) \cdot \frac{\ln(1 + \mu|x_t|)}{\ln(1 + \mu)}$$

- Преобразование сохраняет высокую детализацию для тихих звуков и снижает её для громких



Результаты WaveNet

- Модель училась на закрытом датасете **Google** (английский 25ч, китайский 35ч)
- Звучит естественнее параметрических и конкатенативных систем
- Модель имеет *multi – speaker* реализацию и может генерировать музыку

Speech samples	Subjective 5-scale MOS in naturalness	
	North American English	Mandarin Chinese
LSTM-RNN parametric	3.67 ± 0.098	3.79 ± 0.084
HMM-driven concatenative	3.86 ± 0.137	3.47 ± 0.108
WaveNet (L+F)	4.21 ± 0.081	4.08 ± 0.085
Natural (8-bit μ -law)	4.46 ± 0.067	4.25 ± 0.082
Natural (16-bit linear PCM)	4.55 ± 0.075	4.21 ± 0.071

The Blue Lagoon is a 1980 American romance and adventure film directed by Randal Kleiser



Parametric



Concatenative



WaveNet

WaveRNN, 2018

Проблемы WaveNet:

- Хороший синтез, но очень долгая генерация

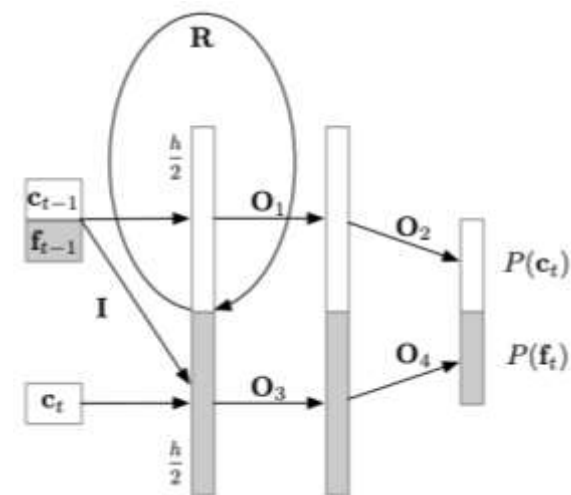
Идея WaveRNN:

- Заменим свёрточную сеть на однослойную *RNN*
- Попытаемся сохранить качество и сделать модель пригодной для использования в реальном времени

Архитектура WaveRNN

- В основе лежит однослойная **GRU**
- Вместо предсказания 65536 классов разобьём предсказания на 2 канала по 8 бит
- Сначала предсказываем старшие 8 бит (**coarse**)
- Зная предсказанные старшие 8 бит, предсказываем младшие (**fine**):

$$p(x_t) = p(c_t) \cdot p(f_t|c_t)$$



Результаты *WaveRNN*

- Модель училась на закрытом датасете (44 ч)
- *WaveRNN* 2048 звучит почти так же, как тяжелый *WaveNet*
- Есть реализации *Sparse WaveRNN* и *Subscale WaveRNN* для быстрой генерации

MODEL	NLL	MOS
WAVENET	5.29	4.51 $\pm$ 0.08
WAVERNN 224	5.67	3.73 $\pm$ 0.09
WAVERNN 384	5.56	4.23 $\pm$ 0.09
WAVERNN 896	5.42	4.37 $\pm$ 0.07
WAVERNN 2048	5.33	4.46 $\pm$ 0.07
SPARSE WR MOBILE	5.52	4.33 $\pm$ 0.08
SPARSE WR 224 / 1536@97.8%	5.48	4.39 $\pm$ 0.07
SPARSE WR 384 / 2048@96.4%	5.42	4.48 $\pm$ 0.07
SUBSCALE WR 1024 (16 $\times$)	5.52	4.30 $\pm$ 0.08
SUBSCALE WR 1024 (8 $\times$)	5.46	4.39 $\pm$ 0.06
FUSED SUBSCALE WR 896 (2 $\times$)	5.45	4.31 $\pm$ 0.08

MelGAN, 2019

Проблемы предыдущих подходов:

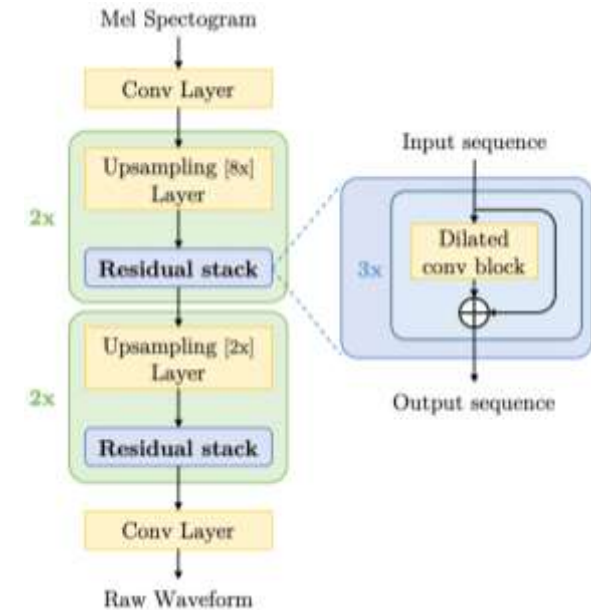
- Хорошее качество, но очень медленный инференс у AR моделей
- Неавторегрессионные модели быстрые, но требуют больших ресурсов для обучения
- Часто требуется дистилляция знаний

Идея MelGAN:

- Будем обучать GAN модель

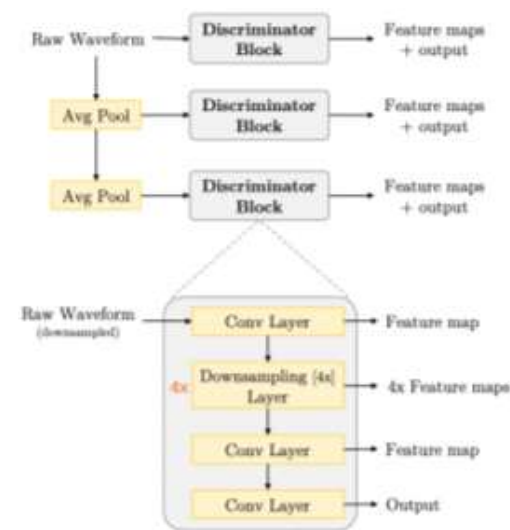
Архитектура генератора *MelGAN*

- Состоит из стека транспонированных свертков для повышения разрешения (*upsampling*) спектрограммы в 256 раз
- После каждого *upsampling* блока идет стек *residual* блоков
- Внутри *residual* блока слои *dilated conv* с $dilation = 1,3$ и 9



Архитектура дискриминатора MelGAN

- Для обучения используется 3 дискриминатора с одинаковой структурой
- Каждый *Discriminator Block* состоит из 4 свёрточных слоёв
- D_1 работает с исходным сигналом
- D_2 работает с сигналом после усреднения *Avg Pool*
- D_3 работает с сигналом после двойного пулинга
- D_1 оценивает мелкие детали, D_3 — глобальную структуру



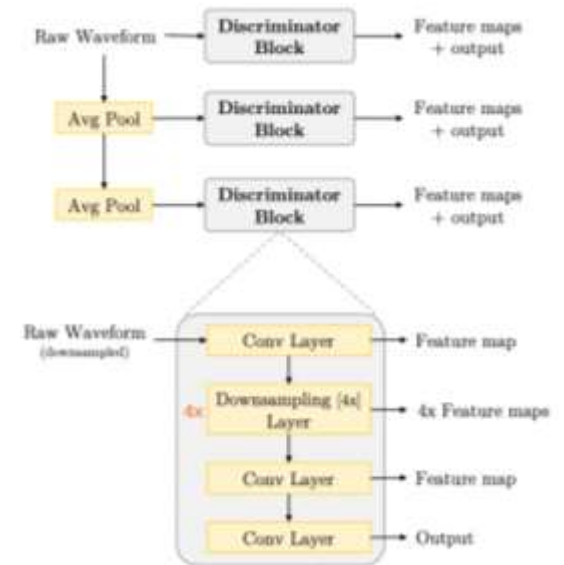
Функция потерь MelGAN

- Дискриминатор выдает не только вероятности *real/fake*, но и промежуточные карты признаков
- Генератор минимизирует сумму двух слагаемых:

$$\min_G \left(\sum_{k=1,2,3} -D_k(G(s)) + \lambda \sum_{k=1,2,3} L_{FM}(G, D_k) \right)$$

где $\lambda = 10$, L_{FM} — *Feature Matching Loss*

- Отсутствует L_1 лосс на разность сигналов



Результаты *MelGAN*

- Модель училась на датасете *LJ Speech*
- Работает в 10 раз быстрее чем *WaveGlow*
- Не требует большого количества ресурсов для обучения
- *MOS* сравним с тяжелыми моделями

Model	MOS	95% CI
Griffin Lim	1.57	± 0.04
WaveGlow	4.11	± 0.05
WaveNet	4.05	± 0.05
MelGAN	3.61	± 0.06
Original	4.52	$\pm$ 0.04

HiFi – GAN, 2020

Проблемы MelGAN:

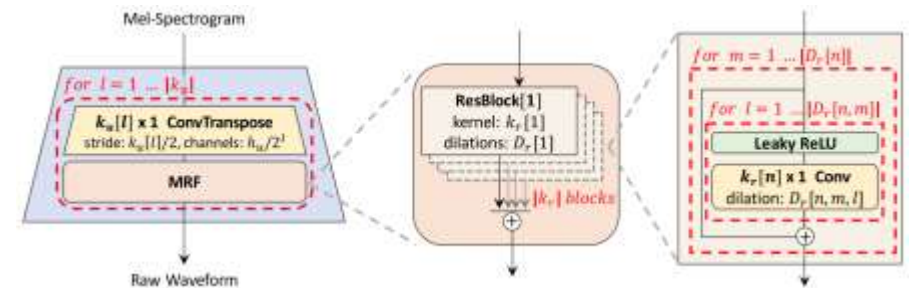
- Эффективен по скорости и памяти, но уступает авторегрессионным моделям по качеству

Идея HiFi – GAN:

- Речь состоит из синусоидальных сигналов с разными периодами
- Моделирование периодических паттернов – ключ к реалистичному звуку

Архитектура генератора *HiFi – GAN*

- Генератор постепенно повышает разрешение (*upsampling*)
- **Multi – Receptive Field Fusion (MRF)** складывает выходы нескольких параллельных *ResBlocks*
- Каждый **ResBlock** использует **dilated convs**, что позволяет смотреть на аудио с разным окном
- Генератор учится фокусироваться и на локальных деталях и на длинных зависимостях



Архитектура дискриминатора *HiFi – GAN*

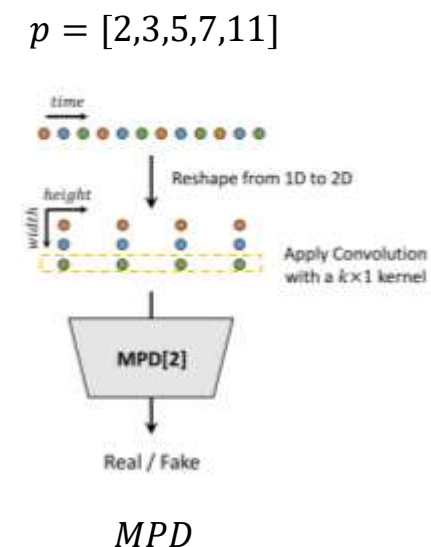
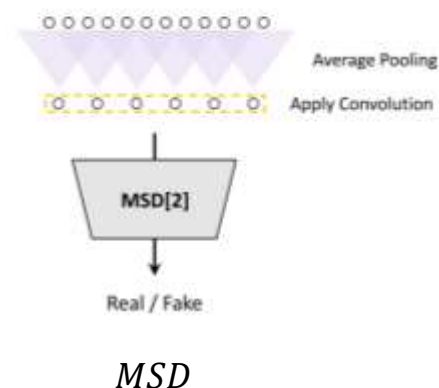
Дискриминатор имеет два вида

Multi – Scale Discriminator (MSP):

- Аналогичен дискриминатору в *MelGAN*
- Сигнал проходит через *Average Pooling*, затем применяются свёртка
- Работает также с исходным сигналом, сжатым в 2 и в 4 раза

Multi – Period Discriminator (MPD):

- Обнаруживает периодические паттерны в сигнале
- Сигнал нарезается и складывается в 2D
- К каждому периоду независимо применяется свёртка $k \times 1$



Функция потерь *HiFi – GAN*

Функция потерь состоит из трех членов:

$$L_G = L_{adv}(G; D) + \lambda_{fm}L_{fm}(G; D) + \lambda_{mel}L_{mel}(G; D)$$

где $\lambda_{fm} = 2$, $\lambda_{mel} = 45$

Результаты *HiFi* – GAN

- Модель училась на датасете *LJ Speech*
- Превзошел результаты лучших моделей
- Есть 3 версии, V3 может работать даже на *CPU*

Model	MOS (CI)	Speed on CPU (kHz)	Speed on GPU (kHz)	# Param (M)
Ground Truth	4.45 (± 0.06)	–	–	–
WaveNet (MoL)	4.02 (± 0.08)	–	0.07 ($\times 0.003$)	24.73
WaveGlow	3.81 (± 0.08)	4.72 ($\times 0.21$)	501 ($\times 22.75$)	87.73
MelGAN	3.79 (± 0.09)	145.52 ($\times 6.59$)	14,238 ($\times 645.73$)	4.26
HiFi-GAN V1	4.36 (± 0.07)	31.74 ($\times 1.43$)	3,701 ($\times 167.86$)	13.92
HiFi-GAN V2	4.23 (± 0.07)	214.97 ($\times 9.74$)	16,863 ($\times 764.80$)	0.92
HiFi-GAN V3	4.05 (± 0.08)	296.38 ($\times 13.44$)	26,169 ($\times 1,186.80$)	1.46

Nothing more was heard of the affair, although the lady declared that she had never instructed Fauntleroy to sell



WaveGlow



MelGAN



HiFi – GAN V1

BigVGAN, 2022

Проблемы прошлых моделей:

- Модели имеют ограниченную ёмкость и плохо работают в условиях вне распределения (*Out-Of-Distribution*)
- Качество резко падает при работе с неизвестными дикторами или языками

Идея BigVGAN:

- Сделаем универсальный вокодер, который будет справляться с любыми сценариями (*zero shot*)

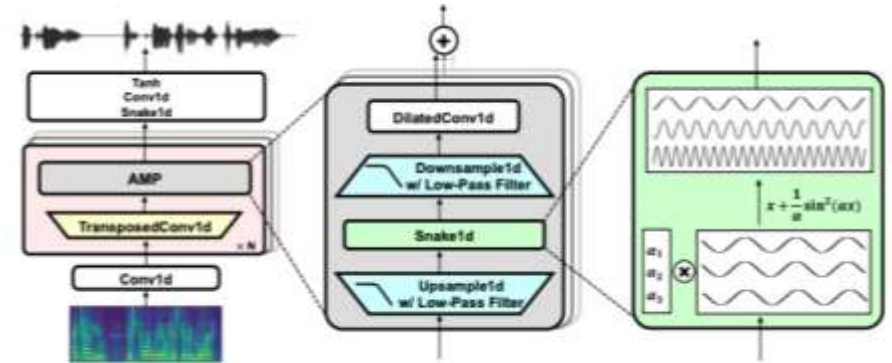
Архитектура *BigVGAN*

Основана на архитектуре *HiFi-GAN* с некоторыми изменениями

- Вместо *LeakyReLU* используется активация *Snake*:

$$f_{\alpha}(x) = x + \frac{1}{\alpha} \sin^2(\alpha x)$$

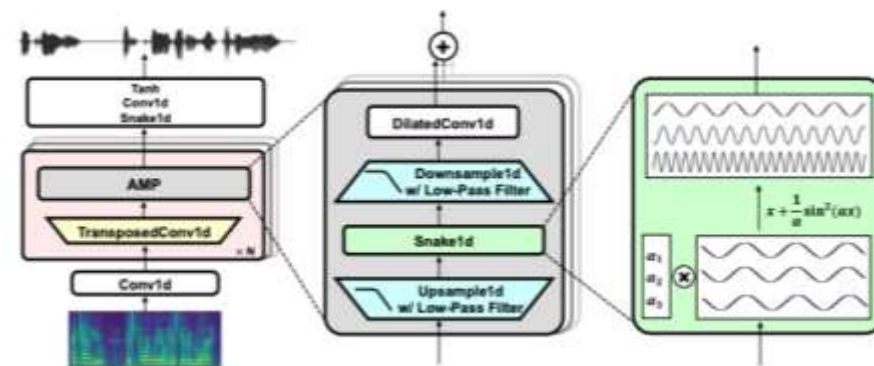
α — обучаемый параметр, что позволяет модели подстраиваться под периодическую структуру сигнала



Архитектура *BigVGAN*

Вместо обычных *Residual* блоков используется модуль *Anti – aliased Multi – Period composition (AMP)*:

- Функция *Snake* может создавать высокочастотные шумы
- Чтобы это избежать, операция активации обёрнута в фильтры
- Разрешение сигнала сначала увеличивается (*upsampling*) с применением низкочастотного фильтра
- Затем применяются нелинейность *Snake*
- Сигнал возвращается к исходному размеру, снова проходя через фильтр



Результаты *BigVGAN*

- Модель училась на *полном* датасете *LibriTTS*
- Превосходит все модели по всем метрикам
- Модель устойчива к новым голосам, языкам и шумам
- На спектрограммах видно меньше артефактов

LibriTTS	M-STFT(↓)	PESQ(↑)	MCD(↓)	Periodicity(↓)	V/UV F1(↑)	MOS(↑)	SMOS(↑)
Ground Truth	-	-	-	-	-	4.40±0.06	4.44±0.06
SC-WaveRNN	2.2358	1.701	1.8854	0.3044	0.8144	3.20±0.11	3.29±0.10
WaveGlow-256	1.3099	3.138	2.3591	0.1485	0.9378	3.84±0.10	3.87±0.10
WaveFlow-128	1.1120	3.027	1.2455	0.1416	0.9410	3.85±0.10	3.89±0.10
HiFi-GAN (V1)	1.0017	2.947	0.6603	0.1565	0.9300	4.08±0.09	4.15±0.09
BigVGAN-base	0.8788	3.519	0.4564	0.1287	0.9459	4.10±0.09	4.20±0.08
BigVGAN	0.7997	4.027	0.3745	0.1018	0.9598	4.11±0.09	4.26±0.08

iSTFTNet, 2022

Проблемы прошлых моделей:

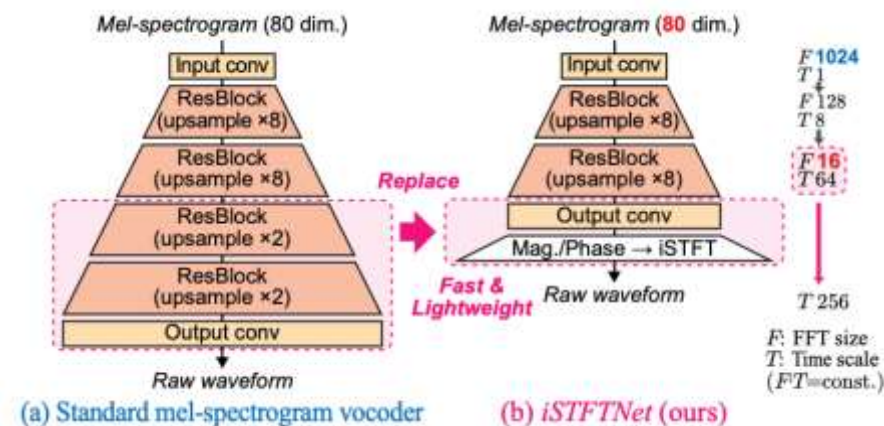
- Работают как чёрный ящик, используя свёртки и *upsampling*
- Вынуждены изучать физику преобразования Фурье
- Игнорируют частотно-временную структуру сигнала и учат её с нуля

Идея iSTFTNet:

- Заменим последние слои нейросети на обратное оконное преобразование Фурье (*inverse Short – Time Fourier Transform, iSTFT*)

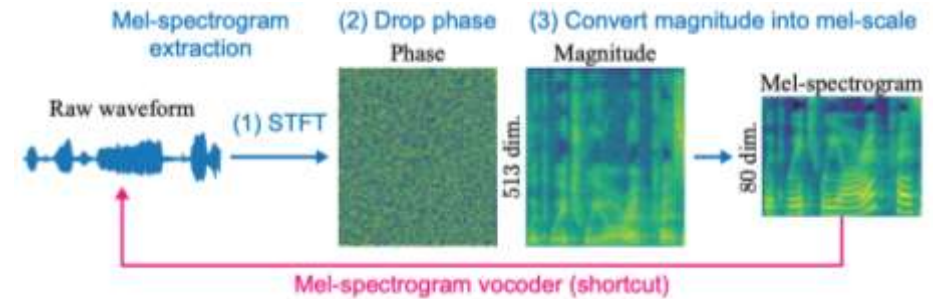
Архитектура *iSTFTNet*

- В качестве основы используется *HiFi-GAN* архитектура
- Удаляются самые последние слои *upsampling*
- Нейросеть выдает амплитуду и фазу
- Эти 2 выхода объединяются и попадают в слой *iSTFT*, который восстанавливает аудио



Обучение *iSTFTNet*

- Обучается *end-to-end*, так как *iSTFT* – дифференцируемая операция
- Функция потерь состоит из 3 частей:
GAN Loss
Mel – Spectrogram Loss
Feature Matching Loss
- Для обучения не нужна истинна фаза, модель учиться восстанавливать её



Результаты *iSTFTNet*

- Нельзя заменить все слои на *iSTFT*
- Модель *C8I*, где мало сверток и ранние *iSTFT*, работает хуже оригинала *V1*
- Модель *C8C8I*, где баланс сверток и *iSTFT*, работает сопоставимо с оригиналом и выигрывает по скорости

No.	Model	MOS↑	cFW2VD↓	Speed↑ (GPU)	Speed↑ (CPU)	# Param↓ (M)
1	Ground truth	4.46 ±0.14	–	–	–	–
2	V1 (original) [12]	4.22 ±0.17	0.020	×143.59 (100)	×1.34 (100)	13.94 (100)
3	V1-C8C8C2I	4.22 ±0.17	0.018	×179.42 (125)	×1.63 (122)	13.80 (99)
4	V1-C8C8I	4.26 ±0.17	0.020	×245.68 (171)	×2.33 (174)	13.26 (95)
5	V1-C8I	3.32 ±0.22	0.073	×609.43 (424)	×7.57 (565)	10.89 (78)
6	V1-C8C1I	3.82 ±0.17	0.033	×326.39 (227)	×3.97 (296)	19.15 (137)
7	V2 (original) [12]	3.91 ±0.17	0.046	×624.47 (100)	×10.39 (100)	0.93 (100)
8	V2-C8C8C2I	3.98 ±0.17	0.038	×732.96 (117)	×13.34 (128)	0.92 (99)
9	V2-C8C8I	3.95 ±0.16	0.042	×1025.46 (164)	×20.37 (196)	0.89 (96)
10	V2-C8I	3.21 ±0.20	0.096	×1720.91 (276)	×68.05 (655)	0.78 (84)
11	V2-C8C1I	3.44 ±0.20	0.071	×1081.37 (173)	×39.14 (377)	1.30 (140)
12	V3 (original) [12]	3.78 ±0.16	0.052	×933.06 (100)	×10.40 (100)	1.46 (100)
13	V3-C8C8I	3.41 ±0.19	0.055	×1517.70 (163)	×21.48 (206)	1.42 (97)
14	V3-C8I	2.89 ±0.17	0.156	×2481.87 (266)	×66.83 (642)	1.28 (87)
15	V3-C8C1I	2.82 ±0.21	0.116	×1925.15 (206)	×41.16 (396)	1.77 (121)
16	MB-MelGAN [24]	3.54 ±0.21	0.078	×1070.95	×17.95	2.54
17	PWG [11]	3.47 ±0.21	0.066	×79.71	×0.70	1.35

Vocos, 2023

Проблемы прошлых моделей:

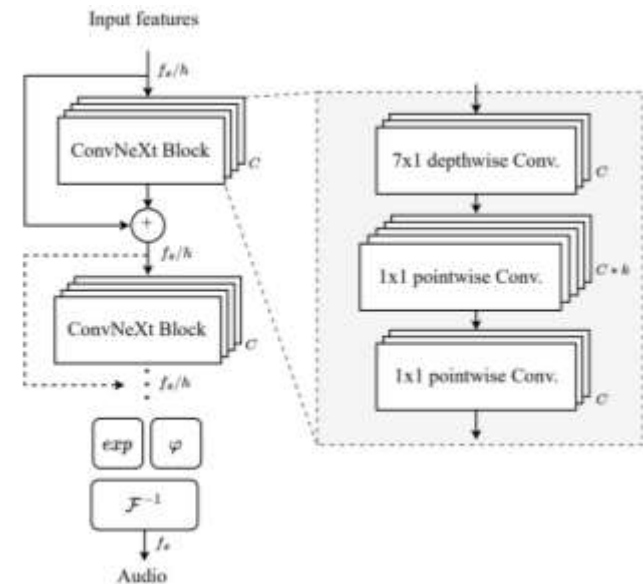
- Почти все модели работают во временной области, делая *upsampling* спектрограммы в аудио
- Игнорируют частотно-временную структуру сигнала и учат её с нуля
- Фазу сигнала сложно моделировать с помощью нейросетей

Идея Vocos:

- Будем генерировать спектральные коэффициенты Фурье и преобразовывать их в звук с помощью *iSTFT*

Архитектура Vocos

- В качестве основы используются **ConvNeXt** блоки
- **Depthwise Conv** обрабатывают пространственную информацию (время)
- **Pointwise Conv** смешивают каналы (частоты)
- Голова предсказывает отдельно амплитуду и фазу
- С помощью **iSTFT** восстанавливается аудио



Обучение Vocos

Функции потерь:

- ***GAN Loss***
- ***Reconstuction Loss (Mel)***
- ***Feature Matching Loss***

Для оценки качества аудио используются:

- ***Multi – Period Discriminator (HiFi-GAN)***
- ***Multi – Resolution Discriminator (UnivNet, BigVGAN)***

Результаты *Vocos*

- Модель обучалась на датасете *LibriTTS*
- Превосходит или соответствует *SOTA* моделям
- В 13 раз быстрее *HiFi – GAN* и в 70 раз быстрее *BigVGAN* на *CPU*
- Устраняет артефакты *GAN* на высоких частотах

	MOS (↑)	SMOS (↑)
Ground truth	3.81±0.16	4.70±0.11
HiFi-GAN	3.54±0.16	4.49±0.14
iSTFTNet	3.57±0.16	4.42±0.16
BigVGAN	3.64±0.15	4.54±0.14
Vocos	3.62±0.15	4.55±0.15

End-to-End Models

VITS, 2021

Проблемы прошлых моделей:

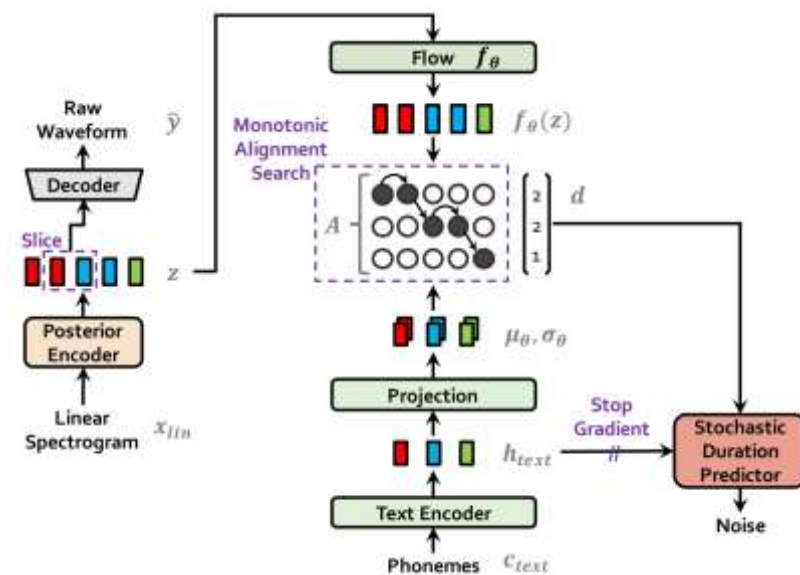
- Традиционные модели работали в 2 этапа акустическая модель → вокодер
- Ошибки накапливались между этапами обучения
- Существующие *end – to – end* модели уступали по качеству двухэтапным системам
- Многие модели используют детерминированный *duration predictor*, из-за чего речь звучит всегда одинаково

Идея VITS:

Объединим *VAE*, *Flow*, *GAN* в одну *end – to – end* модель для генерации естественной и вариативной речи

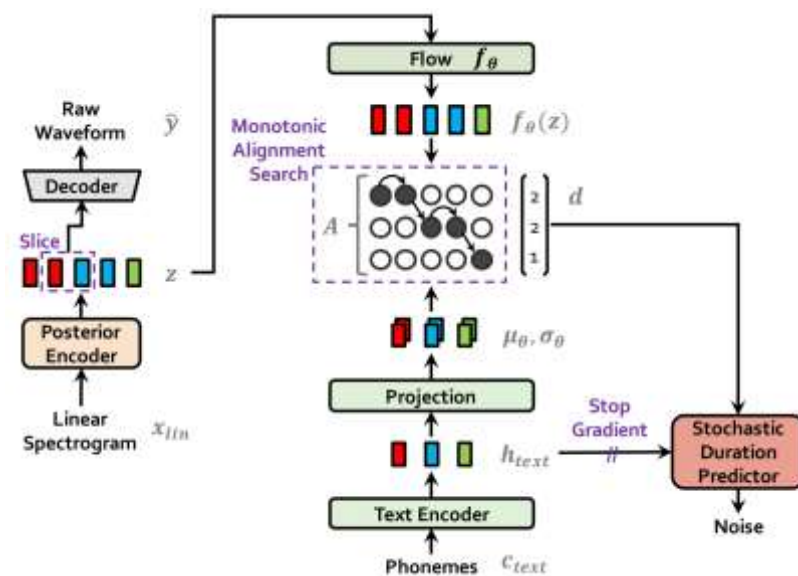
Архитектура VITS

- **cVAE** связывает текст и аудио через скрытое пространство
- **Posterior Encoder** сжимает спектрограмму в скрытый вектор z
- **Decoder** преобразует скрытый вектор z в аудио
- **Normalizing Flow** выступает в роли обучаемого априорного распределения для VAE
- **Stochastic Duration Predictor** предсказывает вероятностное распределение для длительности букв



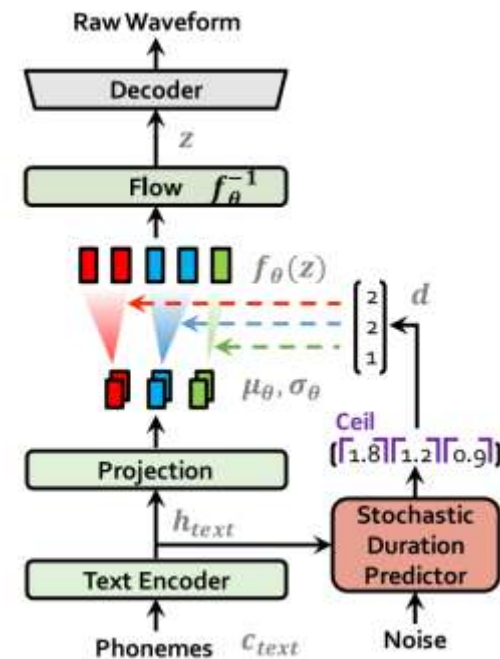
Обучение VITS

- **Posterior Encoder (WaveNet Blocks)** сжимает спектрограмму в скрытый вектор z
- **Text Encoder (Transformer)** обрабатывает фонемы c_{text} и предсказывает μ и σ
- **Normalizing Flow** преобразует скрытый вектор z в простое распределение $N(\mu, \sigma^2)$
- **MAS** находит оптимальное выравнивание между текстом и скрытыми векторами z
- Полученные из MAS длительности фонем используются для обучения *Stochastic Duration Predictor*
- **Decoder (HiFi-GAN)** учится восстанавливать аудио из скрытого вектора z



Генерация VITS

- *Posterior Encoder* выбрасывается
- *Stochastic Duration Predictor* получает на вход скрытые представления h_{text} и шум, чтобы предсказать длительности
- Обратный поток f^{-1} превращает просто распределение в скрытый вектор z
- *Decoder* восстанавливает аудио из вектора z



Результаты VITS

- Модель училась на датасете *LJ Speech*
- Превосходит по качеству лучшие двухэтапные системы
- Работает быстрее авторегрессионных моделей и двухэтапных систем
- Для одного предложения возможна генерация разных интонаций

Model	MOS (CI)
Ground Truth	4.46 (± 0.06)
Tacotron 2 + HiFi-GAN	3.77 (± 0.08)
Tacotron 2 + HiFi-GAN (Fine-tuned)	4.25 (± 0.07)
Glow-TTS + HiFi-GAN	4.14 (± 0.07)
Glow-TTS + HiFi-GAN (Fine-tuned)	4.32 (± 0.07)
VITS (DDP)	4.39 (± 0.06)
VITS	4.43 (± 0.06)

The rainbow is a division of white light into many beautiful colors



Tacotron + HiFi-GAN (ft)



GlowTTS + HiFi-GAN (ft)



VITS

Спасибо за внимание!